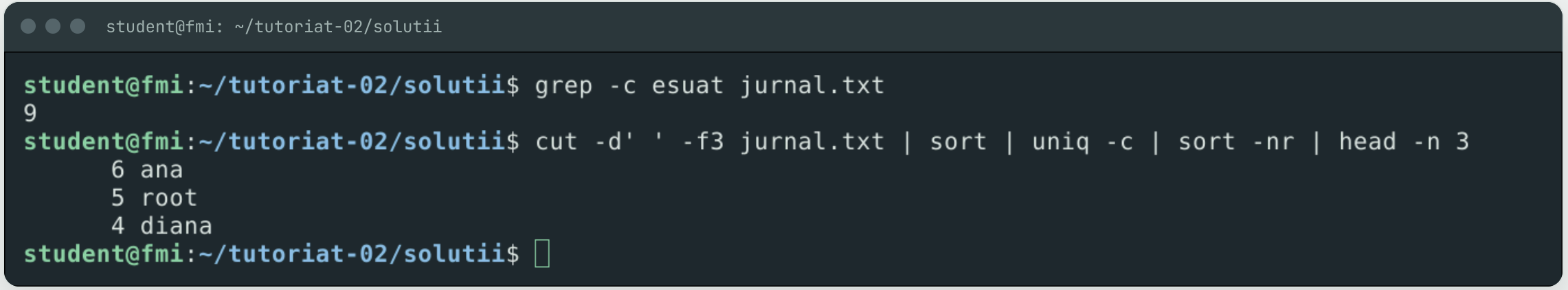
Text care curge prin țevi

Unde scrie un program, cum mutăm ce scrie în fișiere și cum legăm comenzi mici într-o conductă care rezolvă probleme mari.



Captură reală. La finalul orei veți scrie singuri conducta de jos și veți ști ce face fiecare bucată.

PLANUL DE AZI

Cinci întrebări, apoi trecem la terminal

1

Unde scrie un program?

Cele trei fluxuri: `stdin`, `stdout`, `stderr`.

2

Cum trimit rezultatul într-un fișier?

Redirectări: `>`, `>>`, `2>`.

3

Cum leg două comenzi?

Pipe-ul `|` și filosofia UNIX.

4

Ce unelte am pentru text?

Filtre: `grep`, `sort`, `uniq`, `cut`.

5

Când îmi „mănâncă” shell-ul caracterele?

Ghilimele și backslash.

Cuvinte pe care le vom folosi des. Apasă pe unul:

flux · descriptor · stdin · stdout · stderr · redirectare · pipe · filtru · tipar

flux (stream) Un șir de octeți care curge într-o singură direcție: dinspre o sursă (tastatură, fișier) spre o destinație (ecran, fișier, alt program).

Cum împărțim cele 90 de minute



1 · UNDE SCRIE UN PROGRAM

Orice program pornește cu trei „țevi” deja montate

Fiecare are un număr, numit descriptor. Apasă pe un număr.



1 · stdout, ieșirea standard

Aici scrie programul rezultatul, datele. Asta poți trimite într-un fișier cu > sau mai departe printr-un pipe.

În C: stdout, printf(), puts()

```
$ ls date.in lipsa.txt
ls: cannot access 'lipsa.txt': No such file or
directory
date.in
```

venit pe stdout venit pe stderr

2 · REDIRECTĂRI

Mută o țeavă: de pe ecran într-un fișier

Comanda e mereu `ls date.in lipsa.txt`: primul fișier există, al doilea nu. Alege ce adaugi la final.

(nimic) | > rez.txt | 2> err.txt | 2> /dev/null

```
$ ls date.in lipsa.txt > rez.txt
```

1 stdout → rez.txt

2 stderr → ecran

Doar stdout s-a mutat în fișier. Eroarea a rămas pe ecran, ca s-o vezi.

ecranul

```
ls: cannot access 'lipsa.txt': No such file or directory
```

rez.txt

```
date.in
```

2 · REDIRECTĂRI

Ambele țevi în același fișier: 2>&1

Citește-l „2 devine o **copie** a lui 1, așa cum e 1 **acum**”, de la stânga la dreapta.

`> tot.txt 2>&1` | `2>&1 > rez.txt` | `&> tot.txt`

```
$ ls date.in lipsa.txt > tot.txt 2>&1
```

pasul 1 **`> tot.txt`**

1. stdout → `tot.txt`
2. stderr → `ecran`

pasul 2 **`2>&1`**

1. stdout → `tot.txt`
2. stderr → `tot.txt`

Întâi 1 merge în tot.txt. Apoi 2 devine o copie a lui 1, care deja e tot.txt. Totul ajunge în fișier, ecranul rămâne gol.

ecranul

(nimic pe ecran)

tot.txt

```
ls: cannot access 'lipsa.txt': No such file or
directory
date.in
```

2 · REDIRECTĂRI

> golește fișierul întâi, >> adaugă la final

Apasă de mai multe ori pe fiecare și urmărește `f.txt`.

`echo pas > f.txt`

`echo pas >> f.txt`

`sort f.txt > f.txt`

De la capăt

>> a păstrat ce era și a adăugat linia nouă la final.

f.txt

```
pas 1
pas 2
```

```
$ echo pas > f.txt
$ echo pas >> f.txt
```

3 · CONDUCTE

Construiește o conductă, bucată cu bucată

a | b dă stdout-ul lui a ca stdin lui b. Alege o întrebare, apoi adaugă câte o bucată.

Câte încercări au eșuat? | Cine apare cel mai des? | Cine greșește parola? | Câte intrări pe zi?

`cat jurnal.txt` | `cut -d' ' -f3 ?`

```
$ cat jurnal.txt
2026-10-27 08:02:11 ana login ok
2026-10-27 08:05:47 bogdan login ok
2026-10-27 08:06:02 root ssh esuat
2026-10-27 08:06:05 root ssh esuat
2026-10-27 08:06:09 root ssh esuat
2026-10-27 08:15:30 cristi login esuat
... încă 19 linii
```

cat jurnal.txt — 25 linii

Adaugă | cut -d' ' -f3

De la capăt

> „Scrieți programe care fac un singur lucru și îl fac bine, și care lucrează împreună prin text.” · Doug McIlroy, autorul pipe-ului din UNIX. Pipe-ul duce doar stdout: erorile trec pe lângă, direct pe ecran.

4 · FILTRE: GREP

Păstrează doar liniile care conțin un tipar

```
$ grep  -i  -v  -n  -c  linux  date.in
```

Intrarea. Verde = linia trece prin filtru.

```
1  Linux este un nucleu de sistem de operare scris in C.
2  Shell-ul citeste comenzile si porneste programele.
3  Comanda grep cauta un tipar in fiecare linie de text.
4  Comanda sort ordoneaza liniile, iar uniq elimina dublurile vecine.
5  Kernelul Linux gestioneaza procesele si memoria.
6  Un pipe leaga iesirea unei comenzi de intrarea urmatoarei.
7  Mesajele de eroare merg pe stderr, nu pe stdout.
8  Distributiile linux populare sunt Debian, Ubuntu si Fedora.
9  Comanda wc numara liniile, cuvintele si octetii.
10 Redirectarea > suprascrie fisierul, iar >> adauga la final.
11 In LINUX literele mari si mici conteaza.
12 Shell-ul face expandarea mastilor inainte sa porneasca grep.
```

```
$ grep linux date.in
Distributiile linux populare sunt Debian,
Ubuntu si Fedora.
$ echo $?
0  # a găsit ceva
```

^ la începutul tiparului = „linia începe cu”. Încearcă `^comanda` cu `-i`.

De ce scriem mereu `sort | uniq`?

`uniq` compară fiecare linie doar cu cea de dinainte. Dublurile care nu stau una lângă alta îi scapă.

`uniq` | **`uniq -c`** | `sort | uniq -c` | `sort -u`

primele 10 nume din jurnal

```
ana
bogdan
root
root
root
cristi
cristi
ana
diana
bogdan
```

→

după uniq -c

```
      1 ana
      1 bogdan
      3 root
      2 cristi
      1 ana
      1 diana
      1 bogdan
```

uniq -c numără greșit: ana apare de două ori, cu câte 1, pentru că aparițiile nu stau una lângă alta.

4 · FILTRE

Rețeta „cele mai dese valori, primele”

O vei folosi la aproape orice întrebare de tipul „cine apare cel mai des?”.

`cut -d' ' -f3`	ia doar câmpul care te interesează (aici, numele)
`sort`	aduce valorile identice una lângă alta
`uniq -c`	le numără: `6 ana`, `5 root`…
`sort -nr`	cel mai mare număr primul: `-n` numeric, `-r` invers
`head -n 3`	păstrează doar primele trei

De ce -n? Apasă și compară:

`sort` `sort -n` `sort -nr`

9	10
10	100
2 →	2
100	25
25	9

Fără -n, sort compară text, caracter cu caracter, ca în dicționar: „10” începe cu 1, deci vine înaintea lui „9”.

5 · DESPECIALIZARE

Ce primește de fapt programul?

Înainte să pornească o comandă, shell-ul înlocuiește variabilele (`$HOME`) și măștile (`*.c`). Ghilimelele îl opresc.

```
echo $HOME *.c
```

```
echo "$HOME *.c"
```

```
echo '$HOME *.c'
```

```
echo \$HOME \*.c
```

Argumentele primite de program (`argv`):

`argv[0]` `echo` | `argv[1]` `/home/student` | `argv[2]` `exercitiu.c`

```
$ echo $HOME *.c
/home/student exercitiu.c
```

Fără ghilimele: $HOME devine calea, iar *.c devine lista de fișiere .c din folder (o mască, ca în Tutoriatul 01).

`"..."`	înlocuiește `$VAR`, dar păstrează spațiile și `*`
`'...'`	nu înlocuiește **nimic**
`\`	oprește un **singur** caracter

5 · DESPECIALIZARE

Spațiul taie textul în argumente

Fiecare spațiu neprotejat începe un argument nou. Uneori nu e ce voiai.

```
grep ana login jurnal.txt
```

```
grep 'ana login' jurnal.txt
```

```
touch fisier nou.txt
```

argv[0] `grep` argv[1] `ana` argv[2] `login` argv[3] `jurnal.txt`

```
$ grep ana login jurnal.txt
grep: login: No such file or directory
jurnal.txt:2026-10-27 08:02:11 ana login ok
jurnal.txt:2026-10-27 09:01:12 ana sudo ok
jurnal.txt:2026-10-27 09:30:55 diana login ok
... încă 7 linii
```

grep a primit tiparul „ana” și DOUĂ fișiere: login (care nu există) și jurnal.txt. Cum caută în mai multe fișiere, pune numele fișierului în fața fiecărei linii.

> **Regula de reținut:** tiparele pentru `grep` și seturile pentru `tr` între ghilimele **simple**. Variabilele între ghilimele **duble**: `"$fisier"`.

CODURI DE IEȘIRE

Comenzi care decid singure: && și ||

`grep` lasă un cod de ieșire: `0` a găsit, `1` n-a găsit, `2` eroare. Următoarea comandă poate ține cont de el.

`a && b`
b rulează doar dacă a a reușit (0)

`a || b`
b rulează doar dacă a a eșuat

`a ; b`
b rulează oricum

```
grep -q root jurnal.txt && echo gasit
```

```
grep -q windows date.in || echo negasit
```

```
grep windows date.in | wc -l
```

```
$ grep -q root jurnal.txt && echo gasit
gasit
```

grep -q nu afișează nimic, doar lasă codul: 0, a găsit. Deci && rulează echo.

EXERCIȚIUL DE AZI

Scriem în C un mini-`grep` care se poartă frumos

`./exercitiu [-i] [-v] [-n] [-c] <tipar> [fisier]` · patru reguli pentru un program bun de pus într-un pipe:

1. Citește din fișierul primit sau, dacă lipsește, din `stdin`.

   Așa merge și `cat date.in | ./exercitiu linux`.

2. Liniile găsite pe `stdout`. Erorile și statistica pe `stderr`.

   Ca să nu ajungă în pipe sau în fișierul cu rezultate.

3. Cod de ieșire ca la `grep`: 0 găsit, 1 negăsit, 2 eroare.

   Ca să meargă `./exercitiu x f && echo gasit`.

4. Închide cu `fclose()` doar ce a deschis cu `fopen()`.

   Niciodată `stdin`: nu l-a deschis el.

EXERCIȚIUL DE AZI

Rulează soluția și vezi cele patru reguli în acțiune

```
./exercitiu -n Linux date.in
```

```
cat date.in | ./exercitiu -c -i linux
```

```
./exercitiu -v -i linux < date.in 2> /dev/null | wc -l
```

```
./exercitiu windows date.in
```

```
./exercitiu linux lipsa.txt
```

```
$ ./exercitiu -n Linux date.in
1:Linux este un nucleu de sistem de operare scris in C.
5:Kernelul Linux gestioneaza procesele si memoria.
[filtru] 2 din 12 linii potrivite (sursa: date.in)
$ echo $?
0
```

stdout stderr · regula 1: fișier primit ca argument

FIȘĂ RAPIDĂ

Filtrele pe care le folosești azi

Apasă pe o comandă ca să vezi un exemplu pe `jurnal.txt`.

grep caută	**sort** ordonează	**uniq** fără dubluri	**wc** numără	**cut** taie coloane	**tr** înlocuiește caractere
head începutul	**tail** finalul	**tee** ramificație	**less** paginează	**diff** compară	**/dev/null** gaura neagră

```
$ cut -d' ' -f4 jurnal.txt | sort | uniq -c
     11 login
      5 logout
      5 ssh
      4 sudo
```

uniq

Elimină liniile identice vecine. -c le și numără. Aproape mereu după sort.

GREȘELI FRECVENTE

Capcanele clasice cu pipe-uri și redirectări

Apasă pe una: vezi de ce se întâmplă și cum o eviți.

`sort date.txt > date.txt # fișierul rămâne gol!` ⌄

DE CE

Shell-ul pregătește redirectarea înainte să pornească sort: deschide date.txt și îl golește. sort citește apoi un fișier gol.

REZOLVARE

```
sort date.txt > tmp.txt && mv tmp.txt date.txt
```

`cut -d' ' -f3 jurnal.txt | uniq -c # ana apare pe 6 rânduri separate` ›

`make 2>&1 > log.txt # erorile tot pe ecran` ›

`grep: login: No such file or directory` ›

`sort numere.txt → 10 100 2 25 9` ›

`Erorile programului meu apar în fișierul cu rezultate` ›

ÎNCĂLZIRE ÎNAINTE DE KAHOOT

Trei întrebări rapide

Fișierul f.txt are 10 linii. Câte are după echo gata > f.txt?

- 11
- 1
- 10

Pe ce descriptor trebuie să scrie un program mesajele de eroare?

- 0 (stdin)
- 1 (stdout)
- 2 (stderr)

Ce afișează echo '$HOME'?

- /home/student
- $HOME
- nimic

ACUM E RÂNDUL TĂU

Deschide terminalul și urmează pașii

Întâi exercițiile cu pipe-uri din `pipeline.sh`, apoi `TODO`-urile din scheletul `exercitiu.c`.

- [ ] `cd tutoriat-02/schelet` intră în folder
- [ ] `bash pipeline.sh` exercițiile 1–11, cu pipe-uri
- [ ] `nano exercitiu.c` rezolvă TODO 1–9
- [ ] `make && make test` compilează și testează
- [ ] `make pipeline` exercițiile 12–13, cu programul tău
- [ ] `make valgrind` bonus: nimic uitat deschis?

Ce exersezi

Citești din fișier *sau* din `stdin`, separi datele (`stdout`) de mesaje (`stderr`) și întorci un cod de ieșire cu care merg `&&` și `||`.

Bonus: Valgrind

Comentează `fclose(in)`, rulează `make valgrind` și citește ce ți-a rămas deschis. Îl folosim pe larg în Tutoriatul 04.

Te-ai blocat? Soluțiile sunt în `tutoriat-02/solutii/`. Încearcă întâi singur.